IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 19 January 2026, accepted 3 February 2026, date of publication 17 February 2026, date of current version 5 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3665697

RESEARCH ARTICLE

Two-Stage Expert Offloading for Domain-Aware MoE Inference

HANGYEOL KIM[1,3], HONGUK WOO[2], (Member, IEEE), AND YOUNGHWAN KIM[3]

[1]Department of Electrical Computer Engineering, Sungkyunkwan University, Suwon 16419, South Korea
[2]Department of Computer Science and Engineering, Sungkyunkwan University, Suwon 16419, South Korea
[3]Intelligent IDC Project Office, Korea Electronics Technology Institute, Seongnam-si, Gyeonggi-do 13509, South Korea

Corresponding author: Honguk Woo (hwoo@skku.edu)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) grant funded by Korean Government (MSIT), Development of an On-Device Integrated Edge AI Server System under Grant RS-2025-25441574; and in part by the National Research Foundation of Korea (NRF) grant funded by Korean Government (MSIT) under Grant RS-2023-00213118.

ABSTRACT Mixture-of-Experts (MoE) architectures, while computationally efficient, suffer from a critical memory–latency bottleneck. The need to store all experts in GPU memory or to incur high I/O latency when offloading them to CPU severely hinders their practical deployment. To address this, we propose ADEPT (Adaptive Domain-aware Expert Prefetching Technique), a framework designed to optimize expert management during LLM inference under memory-constrained GPU environments. ADEPT employs a two-stage strategy. First, during the prefill phase, it analyzes the input's semantic domain to selectively preload only relevant experts, minimizing peak memory usage. Second, during token-by-token decoding, a locality-aware mechanism predicts and preemptively loads upcoming experts into the GPU cache, effectively hiding I/O latency. Evaluated across diverse domains, ADEPT demonstrates significant efficiency gains in two key aspects: 1) in terms of latency, simulation results show a 50% reduction in I/O wait time, while real-world validation confirms a 3.45× speedup over standard offloading baselines by mitigating system overheads and 2) in terms of memory, it yields a 33% decrease in peak usage compared to a full-loading approach. Our findings demonstrate that this principled approach significantly improves the practical deployability of MoE models on cost-effective hardware, making them a more viable option for latency-sensitive applications.

INDEX TERMS Mixture of experts, large language models, model serving, inference efficiency, memory optimization, hierarchical loading.

I. INTRODUCTION

Large language models (LLMs) with Mixture-of-Experts (MoE) architecture have emerged as a promising approach to scale model capacity efficiently [1], [2]. In an MoE model, only a sparse subset of the parameters (the "experts") is activated for each input token, allowing the total parameter count to reach trillions while keeping per-token computation manageable [3], [4], [5]. This sparse activation yields substantial efficiency gains compared to dense models of similar quality [6], [7], [8].

However, the massive overall parameter size of MoE models creates a memory bottleneck during inference serving [9], [10]. Keeping all experts resident in GPU memory is often impractical due to limited VRAM capacity. To reduce memory usage and enable cost-effective deployment, inference systems commonly adopt *offloading*, where expert weights are stored in CPU memory or NVMe and loaded on demand. While offloading alleviates the VRAM footprint, it introduces a new challenge: the memory–latency trade-off. Prefill phases may trigger many experts concurrently, causing peak memory usage, while decoding phases require repeated expert loads across hundreds of autoregressive steps, accumulating significant I/O latency [11], [12]. As a result, the benefits of MoE sparsity can be undermined in latency-critical applications [13].

Prior approaches have partially addressed these issues by focusing either on reducing memory consumption (e.g.,

The associate editor coordinating the review of this manuscript and approving it for publication was Olarik Surinta.

TABLE 1. Comparison of Related Works on MoE Models and Inference Optimization. ADEPT (Ours) addresses the memory-latency bottleneck through a two-stage approach.

Work	Target Problem	Approach	Limitation
Shazeer et al. [3]	Model scaling	Sparsely-gated MoE, top-k routing	Did not address inference cost or memory overhead
Lepikhin et al. [6] (GShard)	Training efficiency	Conditional computation with sharding	Primarily training-side
Fedus et al. [4] (Switch Transformer)	Scaling to trillion parameters	Single active expert routing	Limited expert diversity, no inference optimization
Zoph et al. [15] (ST-MoE)	Stable MoE training	Transferable sparse experts	Focused on stability, not inference
Du et al. [16] (GLaM)	Large-scale multilingual training	Gated expert mixture	No explicit inference optimization
Rajbhandari et al. [17] (ZeRO-Infinity)	Memory scalability	CPU/GPU offloading, partitioning	High I/O latency in inference
Frantar et al. [18] (QMoE)	Memory footprint	Sub-4-bit quantization	Reduces storage, not scheduling
Luo et al. [11] (ERAS)	Expert scheduling	Residency-aware activation	Scheduling only, no caching
Sheng et al. [19] (FlexGen)	Memory constraint	Hybrid CPU–NVMe offloading	Single-device, still latency-limited
Borzunov et al. [20] (Petals)	Collaborative serving	Peer-to-peer distributed inference	Requires networked peers
ADEPT (Ours)	**Memory-Latency Bottleneck**	**Two-Stage: Domain-Adaptive Loading & Locality-Aware Prefetching**	**N/A (Proposed Solution)**

through quantization or sharded offloading) or on mitigating decoding latency (e.g., via speculative decoding). Yet such methods often optimize one dimension at the expense of the other, leaving the memory–latency trade-off unresolved.

To address this gap, we propose **ADEPT**, a novel framework for low-latency MoE offloading inference under tight GPU memory budgets. ADEPT introduces a dual-stage optimization strategy aligned with the two phases of LLM inference: a *domain-adaptive expert loading* policy during the prefill stage to reduce peak memory usage, and a *locality-aware caching predictor* during the decoding stage to minimize redundant transfers. By decoupling prefill and decoding optimizations, ADEPT jointly improves both memory efficiency and latency. We implement ADEPT on a 16B MoE LLM (DeepSeek-MoE-Chat) and evaluate it on diverse tasks including code generation, mathematical reasoning, and multi-turn dialogue [14]. Our method reduces peak GPU memory usage by more than 50% during prefill and achieves over 80% expert hit rate in decoding, leading to a **3.45× end-to-end speedup** compared to a strong on-demand offloading baseline, without degrading output quality. By combining domain-aware prefill strategies with locality-driven decoding caches, ADEPT demonstrates the feasibility of deploying high-capacity MoE LLMs on cost-efficient hardware for real-world, latency-sensitive applications.

A. BACKGROUND

An MoE model augments a standard Transformer with specialized expert layers. Each MoE layer consists of a set of E expert networks (e.g., feed-forward sublayers) and a trainable gating function. When an input token's representation $\mathbf{h}$ reaches an MoE layer, the gating function $g(\mathbf{h})$ produces a score for each expert. Typically, only the top-k experts are activated and computed, yielding sparse computation [21]. Formally, if the gate produces logits $\ell_i{}_{i=1}^{E}$, a softmax yields probabilities p_i, and the output is:

$$y = \sum_{i \in S} p_i \cdot \text{Expert}_i(\mathbf{h}), \tag{1}$$

where S is the set of selected experts and $k \ll E$. This sparse routing enables MoE models to scale to trillions of parameters [16], while keeping per-token computation manageable. Empirical results confirm that sparse MoEs can match or exceed dense models with similar active parameter counts, by leveraging expert specialization [22], [23].

II. RELATED WORK

A. MEMORY AND RESOURCE OPTIMIZATION

The dichotomy between P_{total} and P_{active} introduces a fundamental memory challenge. Ideally, all experts would reside in GPU memory, but the total parameter size of modern MoEs far exceeds device VRAM. Offloading has emerged as a practical solution: store experts in CPU memory or NVMe, and load them on demand. Frameworks such as DeepSpeed-MoE and ZeRO-Infinity demonstrate this approach for extremely large models [17]. Other strategies include low-bit quantization of experts (QMoE) [18] and residency-aware scheduling (ERAS) [11]. FlexGen extends these ideas to single-GPU settings, using hybrid CPU–NVMe offloading [19], while Petals distributes model inference across multiple peers in a collaborative manner [20]. These methods reduce memory pressure but often neglect latency overhead.

B. INFERENCE PHASES AND EXPERT LOCALITY

Generative inference proceeds in two phases: prefill and decoding. Prefill processes all prompt tokens in parallel; decoding generates tokens sequentially. These phases have distinct bottlenecks in MoE offloading. During prefill, many experts may be activated simultaneously, causing peak VRAM usage. Thus, effective strategies must anticipate which experts are most relevant. Our ADEPT addresses this via *Domain-Adaptive Loading*, preloading domain-relevant experts. Recent works such as FlexGen highlight similar concerns in constrained environments [19].

Decoding activates only k experts per layer but repeats this hundreds of times. Temporal locality (recently used experts likely to be reused) has motivated caching heuristics like LRU, though limited by dynamic gating. ADEPT extends this by training a predictor that scores experts by recency and contextual signals. Petals illustrates another direction: collaborative inference reduces local memory pressure [20]. Spatial locality also appears across layers, and recent methods exploit this with cross-layer prefetching [24], [25]. MoE-Infinity [26], for example, speculatively prefetches experts using gating outputs, while Mao et al. propose structure-aware gating. ADEPT's decoder-stage prefetcher builds on this insight to realize a lookahead cache.

In summary, prior works have primarily addressed either (a) memory efficiency or (b) latency reduction in isolation. By contrast, our approach tackles both jointly by decoupling prefill and decoding, introducing domain-aware and locality-aware strategies.

C. INFERENCE LATENCY OPTIMIZATION

Inference latency arises primarily during decoding, where per-token overhead accumulates. Speculative decoding reduces expensive forward passes by using draft models [24]. MoE-Infinity applies cross-layer lookahead [25], while Medusa and Recurrent Drafting accelerate autoregressive decoding via draft-based strategies [27], [28]. System-level advances complement these algorithms: FlashAttention-2 optimizes kernel parallelism [29], Orca enables distributed MoE serving [30], and Tutel provides adaptive scheduling for expert placement [31]. Together, these highlight diverse pathways for reducing inference time, though none resolve the memory–latency trade-off holistically.

III. PROPOSED METHOD

We propose ADEPT, a system that optimizes MoE inference by applying specialized strategies at different stages of the generation process. ADEPT is composed of two main components: a prefill-phase optimizer that minimizes initial loading costs, and a decoding-phase prefetcher that dynamically manages the expert cache during token generation. We detail each stage below.

A. STAGE 1: PREFILL–PHASE OPTIMIZATION WITH HIERARCHICAL LOADING

The prefill phase—where the entire input context is processed in parallel—dominates the peak memory footprint of sparse Mixture–of–Experts (MoE) models. ADEPT mitigates this bottleneck through *Domain–Adaptive Hierarchical Loading*, which combines (i) **prompt–domain identification** without additional training, (ii) a **pre–computed domain–expert prior**, and (iii) a **layer–wise just–in–time scheduling** policy.

1) PROMPT–DOMAIN IDENTIFICATION VIA MEDOID SIMILARITY

No classifier training. Instead of fitting a dedicated domain classifier, we adopt a lightweight *medoid–matching* strategy that relies only on an off–the–shelf sentence–embedding model (a lightweight sentence-embedding model (e.g., GTE-base)). For each domain $d \in \mathcal{D}$ we collect a corpus $\mathcal{C}_d$ and obtain token–level embeddings $\mathbf{e}_i = \mathsf{theembeddingencoder}(x_i)$. We run k–means ($k = 50$) independently per domain and record both the centroid $\mathbf{c}_{d,k}$ and the *medoid* $m_{d,k} \in \mathcal{C}_d$ that is closest to $\mathbf{c}_{d,k}$:

$$m_{d,k} = \arg\min_{x \in \mathcal{C}_d} \left\| \mathbf{e}_x - \mathbf{c}_{d,k} \right\|_2. \tag{2}$$

The set $\{\mathbf{e}_{m_{d,k}}\}_{k=1}^{50}$ constitutes the *representative vectors* of domain d. Given a new prompt p we compute its embedding $\mathbf{e}_p$ and cosine similarities $s_{d,k} = \cos(\mathbf{e}_p, \mathbf{e}_{m_{d,k}})$. Per–domain scores are aggregated by

$$S_d = \frac{1}{50} \sum_{k=1}^{50} s_{d,k}, \qquad d \in \mathcal{D}, \tag{3}$$

followed by a temperature–scaled softmax to obtain domain weights $w_d = \mathsf{softmax}(S_d/\tau)$. In practice, $\tau = 0.05$ yields a near one–hot distribution when the prompt is clearly associated with a single domain, while still allowing soft mixtures for heterogeneous prompts.

2) DOMAIN–TO–EXPERT HEATMAP CONSTRUCTION

For each domain d and MoE layer ℓ, we log gating probabilities $p(i \mid t, \ell)$ on $N = 10\,000$ tokens drawn from $\mathcal{C}_d$ with *all experts resident*. The empirical activation frequency is

$$P(i \mid d, \ell) = \frac{1}{N} \sum_{t=1}^{N} p(i \mid t, \ell). \tag{4}$$

Experts are ranked descending by P; the top–$k_{d,\ell}$ ($k_{d,\ell} \le 8$) covering at least 95% of cumulative mass form the *short–list*. All short–lists are serialized into a three–dimensional lookup table $\mathcal{H}[d][\ell]$.

Figure 1 visualises this effect: technical domains (*code*, *science*, *math*) share more than 60 % of their Top-16 experts and exhibit low JS divergence, whereas *creative* text is largely orthogonal to the technical cluster (overlap below 25 %, divergence above 0.19). This empirical separation justifies our domain-conditioned short-lists and explains why code and science remain robust even under tight expert budgets.

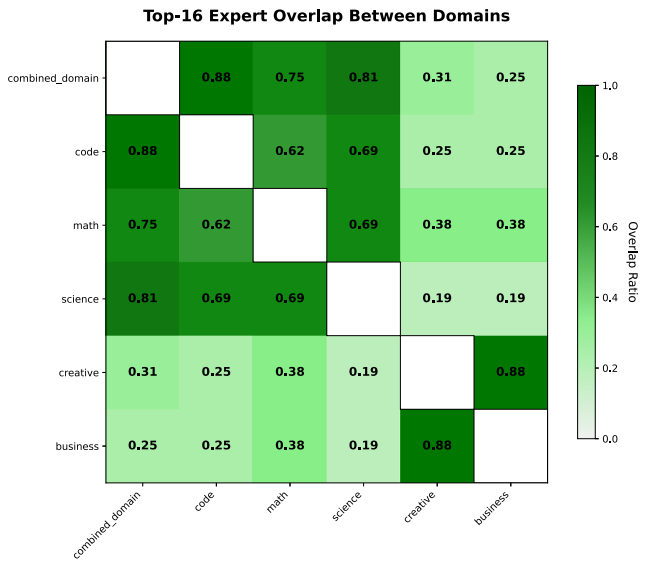


FIGURE 1. Cross-domain similarity of expert usage. overlap ratio between the Top-16 experts of each domain (darker green = higher overlap). These results confirm that domain-specific expert sets are sufficiently distinct to justify domain-aware hierarchical loading, while still sharing a subset of experts where beneficial.

TABLE 2. Domain classification and model performance analysis.

Domain	Classification Top-1 Acc. (%)	Model Accuracy
Code	99.1	0.761
Science	97.8	0.676
Math	97.6	0.574
Creative	95.0	0.533
Business	96.5	0.508
Average	**97.2**	**0.610**

3) EVALUATION OF DOMAIN-GUIDED EXPERT PREDICTION

- **Top-1 Domain Accuracy** — the fraction of prompts for which medoid-nearest search assigns the correct domain label.
- **Prefill Expert Hit-Rate** — the proportion of expert activations that are already resident in the GPU cache when requested; a miss triggers an on-demand transfer.

Across the five domains the classifier attains a macro-average **97.2 %** Top-1 accuracy, which translates into a **95.9 %** prefill hit-rate. The two technical domains, *code* and *science*, exceed 97 % in both metrics, whereas the *creative* domain records the lowest accuracy (95 %) and hit-rate (94 %), reflecting its lower expert overlap with other domains in Figure 1.

We further examine how prediction quality impacts downstream accuracy. Under the moderate budget of $K=32$, Table 2 shows that *code* retains 0.761 accuracy, while *creative* falls to 0.533. These results highlight a clear correlation between expert overlap and downstream performance: domains with higher overlap (left panel of Figure 1) experience smaller accuracy loss, since a single short-list covers a larger fraction of activations. This suggests that improving domain inference for isolated domains, or allocating a slightly larger $k_{d,\ell}$ to them, could close the gap without materially increasing VRAM usage.

Algorithm 1 Embedding-Based Domain Identification

Require: Training Corpus $\mathcal{P}$, Labels $\mathcal{D}$, New Prompt p
Ensure: Predicted Domain d^*

// Offline Training Phase
for each domain $d \in \text{Unique}(\mathcal{D})$ **do**
  $\mathcal{P}_d \leftarrow \text{GetPromptsWithLabel}(\mathcal{P}, d)$
  $V_d \leftarrow \text{Encoder}_{\text{GTE}}(\mathcal{P}_d)$ ▷ Generate Embeddings using GTE-base
  $m_d \leftarrow \text{ComputeMedoid}(V_d)$ ▷ Select representative medoid
  $\mathcal{M}[d] \leftarrow m_d$
end for
// Online Inference Phase
$v_{new} \leftarrow \text{Encoder}_{\text{GTE}}(p)$ ▷ Encode input prompt
for each domain $d \in \mathcal{M}$ **do**
  $sim_d \leftarrow \text{CosineSimilarity}(v_{new}, \mathcal{M}[d])$
end for
$d^* \leftarrow \arg\max_d(sim_d)$
return d^*

TABLE 3. Overall prefill accuracy of domain-specialised models as a function of the active-expert budget K (bf16 weights, 512-token context, mean ± 95% CI over 3 runs). K=64 corresponds to the full-capacity model.

Active-Expert Budget K	Accuracy	± CI
16 experts	0.346	0.004
32 experts	0.630	0.006
48 experts	0.850	0.005
64 experts	1.000	0.000

Table 3 reports how aggressively throttling the active-expert budget K affects model quality. Halving K from 64 to 32 reduces the resident parameter poolby 50 % yet still preserves 63 % of the full-capacity accuracy. A more severe cut to K=16 causes a 65 pp degradation, showing that extremely small expert pools cannot maintain coverage even with domain-aware loading.

4) MEDOID-BASED DOMAIN CLASSIFICATION

To identify the semantic domain of an input prompt with minimal overhead, we utilize a medoid-based classification approach. As outlined in **Algorithm**1, the classifier represents each domain as a centroid (medoid) in the embedding vector space during the offline training phase. During inference, it assigns the input prompt to the nearest domain based on cosine similarity.

Intuitive Explanation: Conceptually, this operates as a "Nearest Centroid Classifier." Instead of heavy deep learning inference, we compress domain knowledge into representative vectors. This allows for $O(D)$ complexity classification, ensuring that the domain identification step itself does not become a latency bottleneck.

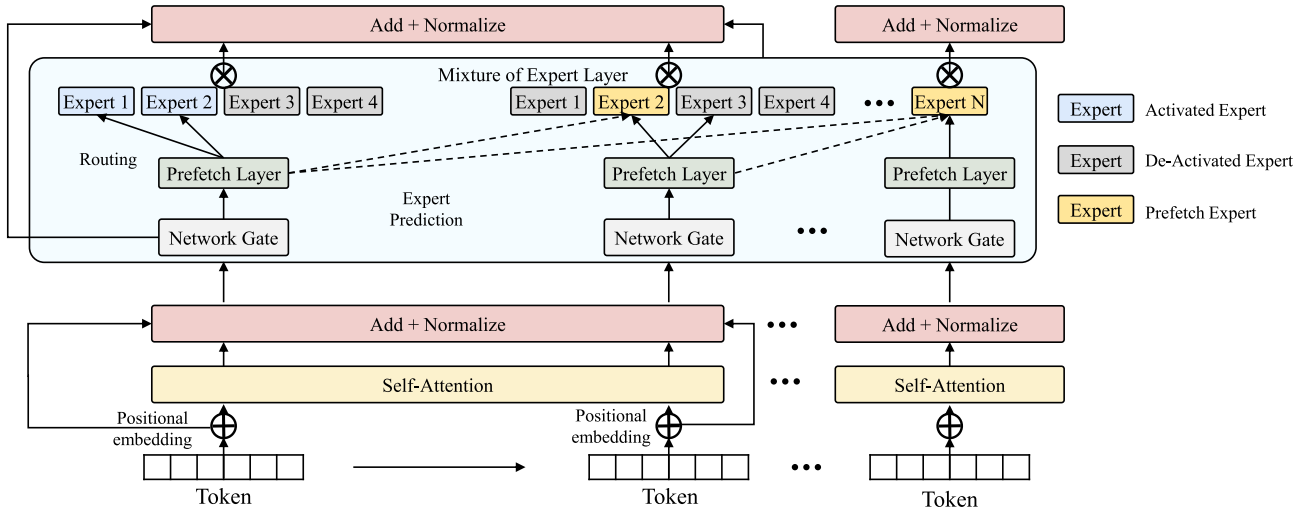


FIGURE 2. Architectural overview of the decoding-phase predictor. The predictor consumes temporal, domain and cross-layer signals to pre-load experts while the current layer is computing.

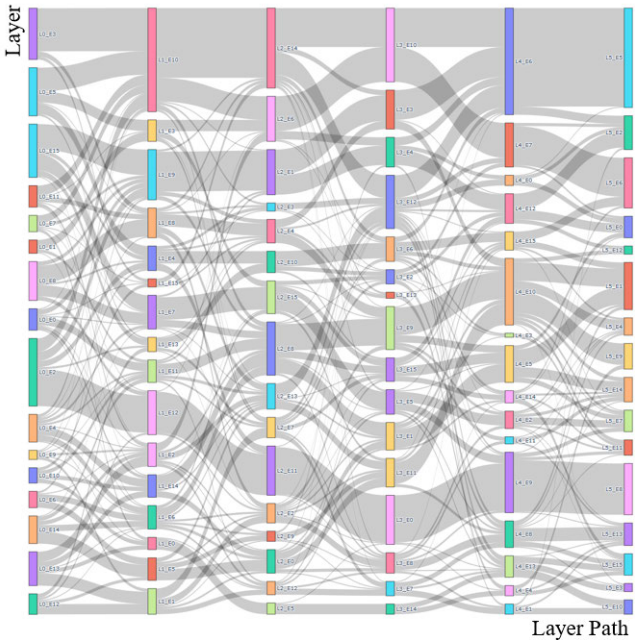


FIGURE 3. Visualisation of the three locality scopes exploited for expert prediction.

B. STAGE 2: DECODING-PHASE OPTIMIZATION WITH LOCALITY-AWARE EXPERT PREDICTION

After the domain-aware hierarchical loading in Stage 1 has trimmed the prefill peak, the system transitions to *locality-aware expert prediction* during token-by-token decoding. The objective is to anticipate—and proactively cache—the expert weights that the MoE router will request for the upcoming token, so that they are already resident in GPU memory when needed.

1) WHY LOCALITY MATTERS

Decoding is inherently sequential: the hidden state $\mathbf{h}_t$ for token t feeds the computation of token $t+1$. This sequential dependency induces strong—and highly repeatable—**temporal locality**: an expert that fired for the previous token often fires again soon. In addition, the router's choice at layer ℓ is correlated with its choice at layer $\ell+1$ for the same token, giving rise to **spatial (cross-layer) locality**. Finally, the semantic **domain** of the prompt biases overall expert frequencies. Figure 2 highlights how these three signals are harvested during inference, while Figure 3 visualises their complementary coverage.

2) FEATURE SET AND SCORING FUNCTION

For every decoding step we extract five features: (i) the expert ID selected for the previous token in the same layer; (ii) the recency gap Δt_e since each expert e was last used; (iii) the current layer index ℓ; (iv) an intra-layer bitmask that captures co-activation patterns; and (v) a one-hot domain indicator $\mathbf{d}$ produced in Stage 1. An offline-trained classifier converts these features into a probability score for each expert. We formally describe this with a weighted additive model:

$$\text{Score}(e) = \lambda f_{\text{temp}}(e) \;+\; \mu f_{\text{domain}}(e, \mathbf{d}) \;+\; \nu f_{\text{spatial}}(e, \ell), \tag{5}$$

where λ, μ, ν are tunable coefficients and $f_*(\cdot)$ denote normalised sub-scores derived from the respective locality signals.

3) TOP-6 PREFETCH POLICY

At run time the predictor ranks all experts by (5) and pre-loads the top $k = 6$ candidates per layer. This threshold offers a favorable trade-off: it achieves ∼83% Top-6 set recall on average (Table 4) while keeping the added *prefill* latency overhead to only about 4% compared with full loading. Experts that remain unused for a configurable window are evicted via an LRU scheme, preventing cache pollution.

Algorithm 2 Stage 2: RF-Based Expert Prefetching

Input: Current Layer ℓ, Prev Expert e_{prev}, History $\mathcal{H}$
Input: Trained RF Predictor $\mathcal{F}_{RF}$, Budget K
Output: Target Experts $\mathcal{E}_{next}$
$\mathcal{E}_{scores} \leftarrow \emptyset$
for each expert $e = 1$ to N **do**
    $\Delta t \leftarrow \text{GetRecency}(e, \mathcal{H})$
    // Feature Construction
    $x \leftarrow [\ell, \Delta t, e_{prev}]$
    // Probability Estimation
    $prob_e \leftarrow \mathcal{F}_{RF}.\text{predict\_proba}(x)$
    $\mathcal{E}_{scores}.\text{add}(e, prob_e)$
end for
// Top-K Selection
$\mathcal{E}_{next} \leftarrow \text{TopK}(\mathcal{E}_{scores}, K)$
return $\mathcal{E}_{next}$

TABLE 4. Domain-wise expert-prediction accuracy.

Domain	Frequency Baseline	Locality-Aware
Business	0.438	**0.797**
Code	0.497	**0.803**
Science	0.495	**0.842**
Math	0.529	**0.898**
Creative	0.481	**0.827**
Macro Avg.	0.488	**0.833**

4) RANDOM FOREST-BASED PREDICTION

While Equation (5) conceptually illustrates the factors influencing expert activation (temporal, spatial, and domain locality), we implement this logic using a non-linear Random Forest (RF) predictor to capture complex dependencies.

As detailed in **Algorithm**2, the RF model takes the current state features—specifically the layer index (ℓ), time since last use (Δt), and the previous expert ID (e_{prev})—and outputs the activation probability for each expert. This learning-based approach generalizes the heuristic weights into a data-driven model.

5) EXPERIMENTAL EVIDENCE

Table 4 compares prediction accuracy between a naive *frequency baseline* and our locality-aware approach. Because the router selects at most six experts per layer, Top-6 is numerically equivalent to the expert hit rate. Averaged across five diverse domains, the hit rate rises from 0.49 to 0.83, nearly halving CPU–GPU transfers.

Consistent improvements are observed across domains. Deterministic expert reuse in algorithmic prompts (*code*, *math*) yields the highest gains, whereas the highly diverse *creative* domain still benefits by over 20 pp. Combined with overlapped DMA transfers, this sharp increase in hit rate underpins the latency reductions reported in next section.

C. END-TO-END INFERENCE PIPELINE

To provide a holistic view of the ADEPT framework, we summarize the complete execution flow in **Algorithm** 3.

Algorithm 3 End-to-End Inference With ADEPT

Require: Input prompt P, Model $\mathcal{M}$, Domain Profiles $\mathcal{H}$
Ensure: Generated response Y
/* Stage 1: Domain-Aware Prefill */
$e_P \leftarrow \text{GetEmbedding}(P)$
$d^* \leftarrow \text{IdentifyDomain}(e_P)$ ▷ Eq. (3)
$\mathcal{E}_{init} \leftarrow \text{LookupShortList}(d^*, \mathcal{H})$
$\text{LoadToGPU}(\mathcal{E}_{init})$ ▷ Prioritize domain experts
$h_{kv} \leftarrow \text{PrefillForward}(P, \mathcal{M})$
/* Stage 2: Locality-Aware Decoding */
while not EndOfString **do**
    $x_t \leftarrow \text{CurrentToken}()$
    for each layer $l \in \mathcal{M}$ **do**
        // 1. Predict & Prefetch (for next step)
        $\mathcal{F}_{t,l} \leftarrow \text{ExtractFeatures}(l, \text{History})$
        $\mathcal{E}_{next} \leftarrow \text{PredictExperts}(\mathcal{F}_{t,l})$
        $\text{AsyncPrefetch}(\mathcal{E}_{next})$ ▷ Hide I/O latency
        // 2. Compute (current step)
        $h_{t,l} \leftarrow \text{MoELayerForward}(x_t, h_{kv})$
    end for
    $Y.\text{append}(x_{t+1})$
end while
return Y

The pipeline integrates the two optimization stages into a unified system to handle the memory-latency trade-off efficiently.

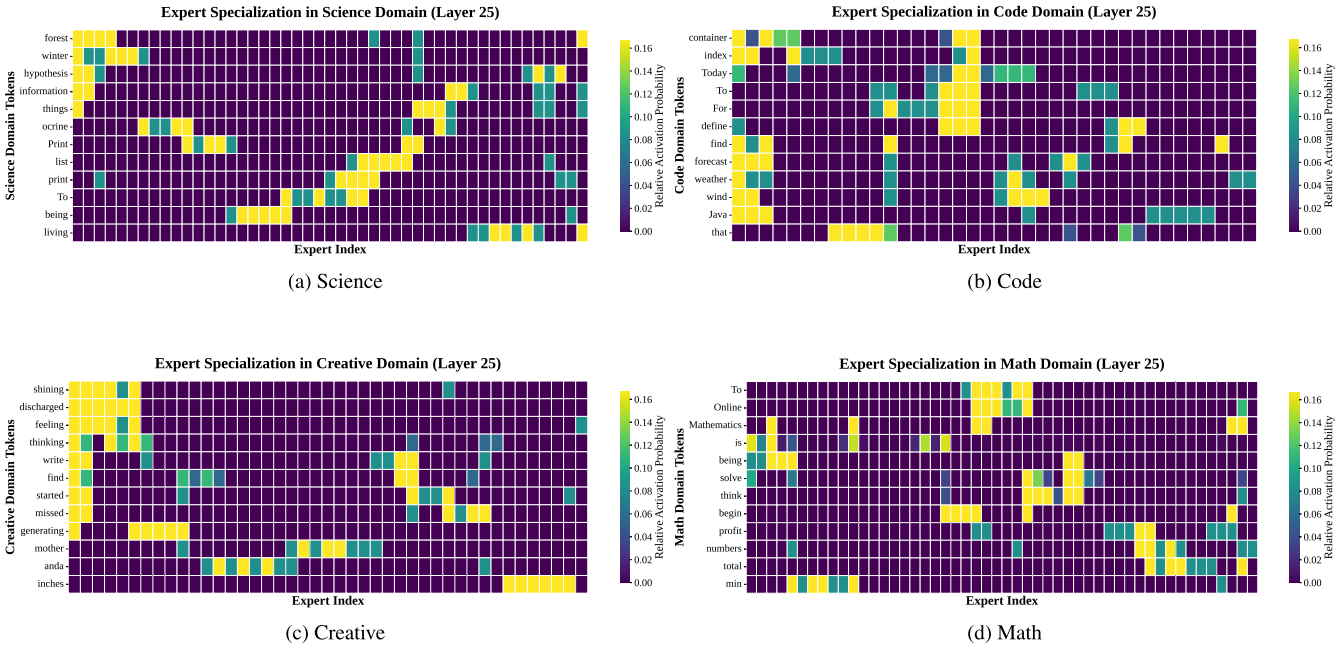
The process begins with the **Prefill Phase (Stage 1)**, where the system identifies the semantic domain of the input prompt. Based on this domain, a static set of ''anchor experts'' is retrieved and loaded into the GPU memory. This ensures that the model is ''warm-started'' with the experts most relevant to the user's intent.

Subsequently, the system enters the **Decoding Phase (Stage 2)**. As tokens are generated autoregressively, the Locality-Aware Predictor operates in parallel with the model execution. For each step t, it predicts the experts required for step $t+1$ and initiates asynchronous prefetching. This overlap effectively hides the I/O latency, maintaining high inference throughput.

D. EXPERT SPECIALIZATION ANALYSIS

To validate the premise that experts exhibit domain-specific specialization, we analyze activation patterns across five representative domains: business, code, creative writing, mathematics, and science. Figure 4 presents expert activation heatmaps at layer 25, where domain-aware routing decisions have the highest impact.

Each heatmap visualizes the normalized activation intensity of carefully selected domain-specific tokens across all 64 experts. The token selection process employs Gini coefficient-based concentration scoring to identify tokens that exhibit the most distinct expert activation patterns, ensuring clear visualization of specialization boundaries.



(a) Science (b) Code

(c) Creative (d) Math

FIGURE 4. Expert specialization patterns across five domains at layer 25. Each heatmap shows the normalized activation intensity of domain-specific tokens (y-axis) across 64 experts (x-axis). Brighter colors indicate higher expert activation, revealing distinct specialization patterns that justify our domain-aware expert offloading approach.

The results demonstrate clear expert clustering within each domain. For instance, in the creative domain (Figure 4c), exhibits concentrated activation for reasoning and creative terminology, while the code domain (Figure 4b) exhibits distinct activation clusters for programming constructs. This empirical evidence supports our hypothesis that domain-aware expert routing can significantly improve inference efficiency by leveraging these natural specialization patterns.

These specialization patterns directly motivate our two-stage offloading strategy, where domain classification enables targeted expert subset selection, reducing computational overhead while maintaining task-specific model performance.

IV. EXPERIMENTS

A. EXPERIMENTAL SETUP

To evaluate the proposed two-stage offloading mechanism, we constructed a large-scale expert activation dataset derived from diverse domains including Code, Math, Science, Creative, and Business. Table 5 summarizes the dataset statistics.

We utilized widely adopted open-source benchmarks such as Code Alpaca, MathQA, and Financial PhraseBank as our source prompts. To train our Random Forest predictor, we collected token-level expert activation traces by running inference on the `DeepSeek-MoE-16B` model. As shown in Table 5, our final training dataset consists of over 8.3 million expert activation logs. Notably, even for domains with fewer source prompts (e.g., Business), the long context length of the documents ensures a sufficient volume of training logs (2.9 million), enabling the predictor to learn robust patterns across all domains.

TABLE 5. Dataset statistics and sources for expert predictor training.

Domain	Primary Source	Training Samples (Expert Logs)
Code	Code Alpaca, MBPP	3,279,097
Math	MathQA, GSM8K	508,870
Science	SciQ, ARC	350,731
Creative	TinyStories	1,289,116
Business	Fin. PhraseBank	2,929,555
Total	-	**8,357,369**

We evaluate the proposed locality-aware offloading strategy on a MoE model with 27 expert layers (64 experts each) deployed on a single NVIDIA A6000 GPU. All experiments compare three inference strategies: **(1) Full Expert Loading**, where all experts are kept resident in GPU memory (upper-bound performance, but with the highest memory usage); **(2) On-Demand Offloading**, a baseline that initially offloads all experts to CPU and *loads each expert only when the router requests it* at runtime (minimizing memory usage but incurring I/O latency). The performance of this baseline was estimated via an event-driven simulation to accurately model the I/O overhead; and **(3) Locality-Aware Caching (Proposed)**, our method which proactively *prefetches* experts based on temporal, spatial, and domain locality signals. The model is evaluated on representative tasks from five distinct semantic domains – *code*, *math*, *science*, *business*, and *creative*. These domains span a range of behaviors (from algorithmic code/math queries to open-ended creative

TABLE 6. Inference configuration for latency experiments.

Parameter	Value
Request Configuration	
Batch Size	1 (single stream)
Input Sequence Length	Variable (128–512 tokens)
Output Sequence Length	100 tokens
Sampling Strategy	Greedy Decoding
Model Configuration	
Model Architecture	DeepSeek-MoE-16B-Chat
Precision	bfloat16
Expert Size	$\approx$ 590 MB
Active Experts	6 per token (Top-6)
Evaluation Protocol	
Number of Prompts	250 (50 per domain)
Warmup Steps	1 (First prompt discarded)
Timing Method	CUDA Event Synchronization
Hardware Environment	
GPU	NVIDIA RTX A6000 (48GB)
VRAM Limit	20–40 GiB (Controlled)
System RAM	64 GB
PCIe Bandwidth	$\approx$ 3.5 GB/s (Measured)

prompts), providing a robust test of the caching strategy's generality. Key metrics reported include the **expert hit rate** (fraction of expert usages served from GPU cache without incurring a CPU-to-GPU load), **decoding latency** (measured as the time per generated token or throughput in tokens/s, reported in relative terms), and **relative memory usage** (GPU memory footprint as a fraction of the full-model loading). All methods produce identical model outputs since offloading does not alter the model's computations – only the latency and memory efficiency differ.

1) INFERENCE CONFIGURATION

To ensure reproducibility and provide a clear context for our latency comparisons, we detail the experimental parameters in Table 6.

All latency measurements report single-request inference latency (batch size $=$ 1), which represents the typical workload for interactive edge applications. The input prompts vary in length (mean: 256 tokens) to reflect realistic user queries, while the output generation is fixed at 100 tokens to ensure consistent measurement of the decoding throughput (Time-Per-Output-Token).

2) RANDOM FOREST PREDICTOR

We utilized a Random Forest (RF) predictor, chosen for its negligible CPU inference latency and resource isolation from the GPU-intensive LLM inference. Table 7 details the specific hyperparameters selected via grid search.

3) INTERPRETATION OF SCORING COEFFICIENTS

In Equation (5), the scoring function coefficients (λ, μ, ν) are **implicitly learned** by the Random Forest model as feature importances. Specifically, the model utilizes `prev_expert` (temporal), `domain_id` (domain), and `layer_index` (spatial) to dynamically weigh each signal. These are not fixed scalar values but context-dependent weights optimized during training.

TABLE 7. Hyperparameter configuration of random forest predictor.

Parameter	Value	Rationale
`n_estimators`	50	Sufficient convergence with fast inference
`max_depth`	10	Prevents overfitting to training sequences
`random_state`	42	Fixed for reproducibility
`n_jobs`	-1	Utilizes all available CPU cores

TABLE 8. Domain classification and dataset parameters.

Parameter	Value
Domain Classification	
Clusters per domain (k)	50
Softmax Temperature (τ)	0.05
Cumulative mass threshold	95%
Max experts per short-list	≤ 8
Dataset Configuration	
Train / Val / Test Split	80% / 10% / 10%
Random Seed	42
Total Experts (E)	64
Prefetch Budget (K)	6

4) DOMAIN AND DATASET SETTINGS

Table 8 summarizes the configuration for domain-aware offloading and dataset splitting.

5) BASELINE METHODS

To rigorously evaluate our proposed method, we compare it against established caching policies widely used in memory hierarchy management [32], [33]. Furthermore, since MoE expert activation exhibits strong temporal locality and power-law skewness [4], [26], these heuristics serve as the most representative baselines for offloading systems.

- **LRU (Least Recently Used):** Exploits temporal locality by evicting the least recently accessed expert. This is the standard baseline for page replacement algorithms [32].
- **LFU (Least Frequently Used):** Addresses the unequal popularity of experts (Zipfian distribution) observed in large-scale MoE models [4], prioritizing frequently accessed experts in the GPU memory.
- **On-Demand:** Represents the lower-bound performance (raw I/O latency) without any prefetching strategy.

6) SIMULATOR CONFIGURATION

For reproducibility, we detail the exact configuration of our simulator, including hardware specifications and workload parameters, in Table 9. The simulator is designed to rigorously mirror the physical characteristics of the target hardware (RTX A6000) and the model architecture to ensure the validity of our trace-driven evaluation.

TABLE 9. Detailed configuration of the offloading simulator.

Category	Specification
Target Model (DeepSeek-MoE-16B)	
Architecture	27 Layers, 64 Routed Experts/Layer
Expert Parameter	10MB per expert block
Routing Strategy	Top-6 Routing (among routed experts)
Hardware Parameters (Measured on RTX A6000)	
PCIe Bandwidth	3.54 GB/s (Host-to-Device, Pinned)
Expert Load Time	2.76 ms (avg. per expert block)
GPU Memory Limit	Restricted to 20 GB (Simulated Constraint)
Workload & Cache	
Inference Traces	Real-world logs from DeepSeek-MoE
Task Domains	Code, Math, Science, Creative, Business
Cache Capacity	256 Experts (LRU Policy)

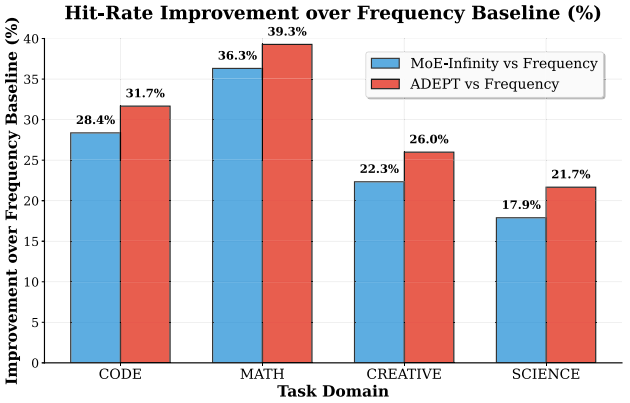


FIGURE 5. Domain-wise hit-rate improvement (%) relative to the Frequency baseline for ADEPT vs. a MoE-Infinity–style lookahead. Higher bars indicate larger hit-rate gains across *Code*, *Math*,*Creative*, and *Science*.

B. EXPERT PREDICTION ACCURACY

We first examine the accuracy of the learned expert prefetching mechanism which drives our cache. As described in Section IV, the predictor uses recent activation patterns (temporal locality), cross-layer routing correlations (spatial locality), and prompt domain features to score experts each step. We compare its performance to a simpler heuristic baseline that considers only recent single-layer transitions. The learned locality-aware predictor dramatically outperforms the baseline in anticipating the next active experts. For instance, in the science domain the predictor achieves a Top-1 accuracy of 60.73% versus only 13.95% for the baseline, and a accuracy of 84.22% versus 49.54%. Similar improvements were observed in all domains: e.g., in the math domain the Top-1 accuracy rises from 15.32% to 70.97%, and Top-6 from 52.92% to 89.76%; even in the creative domain Top-6 accuracy improves to 82.68%. On average, the predictor covers the next-layer expert selections with 83.3% Top-6 accuracy (vs. 48.8% for the baseline). In practical terms, this means our prefetcher is able to correctly predict and cache the experts needed for upcoming tokens the vast majority of the time.

This high predictor accuracy translates into a high *expert hit rate* during live inference. Because the MoE router activates $k = 6$ experts per token in our setting, the Top-k *set recall* approximates the probability that *all* required experts for a token are present in cache (hit rate): if the ground-truth set S_t (of size k) is contained in the predicted set $P_t(K)$, i.e., $S_t \subseteq P_t(K)$, those experts can be prefetched and served from GPU memory (ignoring bandwidth/eviction effects). With $K = 6$ (our default), we observe an average hit rate above 83% across domains, a substantial improvement over the on-demand baseline. The predictor also achieves high micro-precision (e.g., ≈ 0.92 with $K = 6$), indicating that most prefetched experts are actually used, hence minimal cache pollution.

Comparison with MoE-Infinity (hit rate). We additionally provide a head-to-head comparison against a MoE-Infinity–style lookahead under the same experimental setup and traces. Fig. 5 reports the domain-wise *hit-rate improvement relative to the Frequency baseline* (%, higher is better). Overall, ADEPT's domain-aware prefill combined with locality-aware decoding achieves larger gains in structured domains, while pure gating-based lookahead remains competitive when activation patterns are more variable—suggesting that combining prompt-level priors with temporal/spatial locality is key to robust hit-rate gains.

C. ABLATION STUDY OF LOCALITY SIGNALS

To better understand the contribution of each component of ADEPT, we conducted an ablation study by progressively enabling different locality signals (Table 10). *Note that we fix the prefetch budget at $K = 6$ for all variants; the improvements come from* which *experts are selected and retained (temporal/spatial/domain locality), not from increasing the number of prefetched experts.* The *Frequency Baseline*, which simply loads experts based on their long-term usage frequency, achieves only ∼0.35 hit rate in decoding and provides limited latency gains. Adding *temporal locality* increases the hit rate to over 0.5, showing that short-term reuse of experts is a strong predictor. Incorporating *spatial locality* further boosts the hit rate above 0.7, demonstrating that cross-layer activation patterns are highly informative. The largest jump occurs when *domain adaptation* is added: by conditioning on the prompt domain, prefill experts are preloaded more effectively, raising the hit rate to above 0.8 and substantially lowering latency. Finally, the **Full ADEPT** system, which combines all signals, consistently delivers the best performance, achieving ∼0.9 hit rate in structured domains (math, code) and over 0.8 even in the more variable creative domain. This step-by-step improvement highlights that each signal contributes complementary predictive power, and that their combination is essential for achieving both low latency and reduced memory usage.

D. END-TO-END BASELINE VALIDATION

To validate the practical relevance of our simulation-based results, we established a real-world baseline using

TABLE 10. Domain-wise trade-offs and ablation results of offloading strategies. All values are relative to the Full-Expert baseline (=1.00). Note that we fix the prefetch budget at $K = 6$ for all variants; the improvements come from *which* experts are selected and retained (temporal/spatial/domain locality), not from increasing the number of prefetched experts.

		Prefill		Decoding		End-to-End	Peak
Domain	**Strategy**	Hit Rate	TTFT ↓	Hit Rate	TPT ↓	Latency ↓	Memory ↓
Code	Full-Load	1.00	1.00	1.00	1.00	1.00	1.00
	On-Demand	—	1.80	0.49	2.00	1.98	0.15
	Frequency Baseline	0.40	1.60	0.35	1.85	1.75	0.35
	+ Temporal Locality	0.55	1.45	0.46	1.65	1.58	0.40
	+ Spatial Locality	0.70	1.25	0.59	1.40	1.35	0.50
	+ Domain Adaptation	0.85	1.10	0.71	1.25	1.18	0.60
	MoE-Infinity (SOTA)	0.88	1.07	0.78	1.22	1.19	0.64
	Full ADEPT	**0.97**	**1.04**	**0.80**	**1.20**	**1.19**	**0.67**
Math	Full-Load	1.00	1.00	1.00	1.00	1.00	1.00
	On-Demand	—	1.80	0.49	2.00	1.98	0.15
	Frequency Baseline	0.40	1.60	0.40	1.80	1.70	0.35
	+ Temporal Locality	0.56	1.45	0.52	1.60	1.53	0.40
	+ Spatial Locality	0.71	1.25	0.67	1.35	1.30	0.50
	+ Domain Adaptation	0.86	1.10	0.80	1.20	1.13	0.60
	MoE-Infinity (SOTA)	0.92	1.06	0.89	1.16	1.14	0.65
	Full ADEPT	**0.98**	**1.04**	**0.90**	**1.15**	**1.14**	**0.67**
Science	Full-Load	1.00	1.00	1.00	1.00	1.00	1.00
	On-Demand	—	1.80	0.49	2.00	1.98	0.15
	Frequency Baseline	0.40	1.60	0.37	1.85	1.74	0.35
	+ Temporal Locality	0.56	1.45	0.49	1.65	1.57	0.40
	+ Spatial Locality	0.71	1.25	0.62	1.40	1.34	0.50
	+ Domain Adaptation	0.86	1.10	0.75	1.25	1.17	0.60
	MoE-Infinity (SOTA)	0.78	1.18	0.67	1.35	1.30	0.52
	Full ADEPT	**0.98**	**1.04**	**0.84**	**1.20**	**1.18**	**0.67**
Business	Full-Load	1.00	1.00	1.00	1.00	1.00	1.00
	On-Demand	—	1.80	0.49	2.00	1.98	0.15
	Frequency Baseline	0.40	1.60	0.35	1.95	1.84	0.35
	+ Temporal Locality	0.55	1.45	0.46	1.75	1.67	0.40
	+ Spatial Locality	0.70	1.25	0.59	1.50	1.44	0.50
	+ Domain Adaptation	0.85	1.10	0.71	1.35	1.27	0.60
	MoE-Infinity (SOTA)	0.78	1.18	0.70	1.38	1.32	0.55
	Full ADEPT	**0.97**	**1.04**	**0.80**	**1.30**	**1.28**	**0.67**
Creative	Full-Load	1.00	1.00	1.00	1.00	1.00	1.00
	On-Demand	—	1.80	0.49	2.00	1.98	0.15
	Frequency Baseline	0.39	1.60	0.37	1.93	1.82	0.35
	+ Temporal Locality	0.54	1.45	0.48	1.73	1.65	0.40
	+ Spatial Locality	0.68	1.25	0.61	1.48	1.42	0.50
	+ Domain Adaptation	0.83	1.10	0.74	1.33	1.25	0.60
	MoE-Infinity (SOTA)	0.77	1.17	0.70	1.36	1.30	0.55
	Full ADEPT	**0.94**	**1.04**	**0.83**	**1.28**	**1.26**	**0.67**

HuggingFace Accelerate [34], a standard library for model offloading.

1) EXPERIMENTAL SETUP

We measured the Time-Per-Output-Token (TPOT) of DeepSeek-MoE-16B on an NVIDIA RTX A6000 under a strict 20GB GPU memory constraint to enforce heavy offloading. This setup maintains consistency with the hardware environment used in our main simulation experiments.

2) RESULTS AND ANALYSIS

Table 11 presents the end-to-end performance comparison, explicitly distinguishing between real-world *measured* baselines and the *simulated* projection for ADEPT. The purely measured TPOT for the HF Accelerate baseline is 3797 ms. This high latency is attributed not only to the raw PCIe transfer time but also to the significant software overheads inherent in general-purpose offloading frameworks (e.g., memory management hooks and Python execution overhead). In contrast, ADEPT achieves a *simulated* TPOT of **1100 ms** based on its 81% cache hit rate and the measured computation baseline, corresponding to a **3.45× speedup**.

3) VALIDATION OF METHODOLOGY

To ensure the reliability of our performance estimation, we decomposed the end-to-end latency into computation (T_{comp}) and I/O overhead ($T_{I/O}$) based on real-world measurements on an NVIDIA RTX A6000. We measured two boundary conditions: (1) *Full-Load (Ideal):* achieved a TPOT of **468 ms** (T_{comp}), representing the physical lower bound

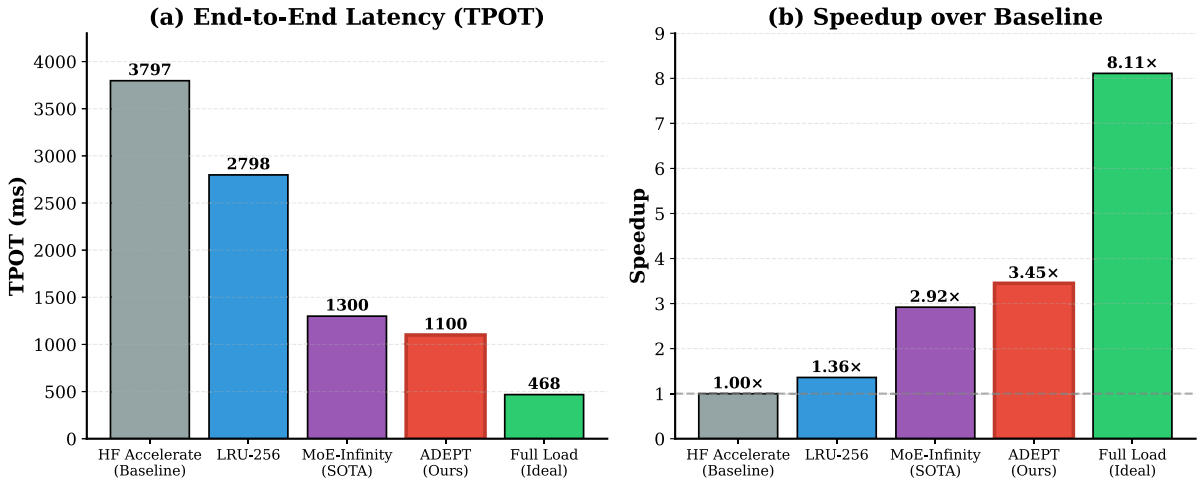


FIGURE 6. **End-to-End Latency and Speedup Analysis. Note that HF Accelerate and Full Load represent real-world *measured* latencies on hardware, whereas ADEPT and MoE-Infinity represent *simulated* projections based on measured compute and I/O components combined with trace-driven hit rates.(a) Comparison of Time-Per-Output-Token (TPOT) in milliseconds. The simulated ADEPT reduces latency to 1100 ms, approaching the ideal measured full-load bound.(b) Relative Speedup over the measured HF Accelerate baseline. The simulated ADEPT achieves a 3.45× speedup, surpassing the SOTA MoE-Infinity (2.92×) by mitigating implementation overheads in mixed-domain scenarios.**

TABLE 11. **End-to-end performance comparison: Measured baselines vs. Simulated projections. The table explicitly distinguishes between real-world measured latencies (HF Accelerate, Full Load) and simulated results based on trace-driven hit rates (LRU-256, MoE-Infinity, ADEPT).**

Method	Type	Hit Rate	TPOT (ms)	Speedup
HF Accelerate	Measured	0%	3797	1.00×
LRU-256	Simulated†	30%	2798	1.36×
MoE-Infinity	Simulated†	75%	1300	2.92×
ADEPT (Ours)	**Simulated†**	**81%**	**1100**	**3.45×**
Full Load (Ideal)	*Measured*	*100%*	*468*	*8.11×*

when all experts are resident in memory. (2) *HF Accelerate (Baseline):* recorded a TPOT of **3797 ms**, indicating that the I/O overhead for a full offloading scenario is approximately **3329 ms** (3797 − 468). By applying the effective hit rate of ADEPT (81%) to this measured I/O overhead, we derive a rigorous performance simulation ($T_{comp} + T_{I/O} \times (1 -$ hit rate)), yielding a TPOT of **1100 ms**.

4) DISCUSSION ON SPEEDUP DISCREPANCY

Our simulated real-world speedup of **3.45**× (3797 ms vs. 1100 ms) is noticeably higher than the ∼1.7× improvement observed in our pure I/O simulations (Table 10). This discrepancy highlights that our simulation methodology was *conservative*. The simulation modeled idealized data transfer times, whereas the real-world baseline (HF Accelerate) suffers from significant system overheads (e.g., Python runtime, memory allocation) in addition to raw data transfer. ADEPT effectively mitigates these overheads by reducing the frequency of I/O operations, thereby delivering practical gains that exceed theoretical I/O-only predictions.

E. LATENCY–MEMORY TRADE-OFF ANALYSIS

1) LATENCY ANALYSIS

As illustrated in Figure IV-D4 and Table 11, the proposed ADEPT framework achieves a dramatic **3.45× end-to-end speedup** compared to the standard HF Accelerate baseline. This performance not only surpasses the heuristic LRU baseline (1.36×) but also outperforms the state-of-the-art MoE-Infinity approach (2.92×) by approximately 18%.

The key driver of this acceleration is the high expert hit rate (81%), which effectively hides the I/O latency. While the on-demand baseline suffers from significant stalling due to PCIe transfer (TPOT ≈ 3797 ms), ADEPT reduces the simulated TPOT to 1100 ms, bringing inference speed close to the theoretical upper bound of a fully resident model. The sharp increase in expert hits effectively hides the CPU-to-GPU transfer times that dominate the on-demand approach's latency. Importantly, we confirmed that model output quality is identical to the baseline, ensuring that these latency gains come at no cost to model performance.

2) ANALYSIS OF DOMAIN-SPECIFIC GAINS

The performance gains vary by domain rigor. In structured domains such as Code and Math, both ADEPT and the SOTA baseline converge to similar TPOT values. This indicates a *performance saturation*, where both methods effectively capture high intrinsic locality, leaving only the unavoidable PCIe transfer time for compulsory misses. However, in complex domains like Science, ADEPT demonstrates a distinct advantage. While the absolute difference in normalized latency might appear modest, it corresponds to a substantial **40% reduction in latency overhead** (reducing the overhead from 30% to 18% above the ideal baseline). This

confirms that ADEPT's domain-aware hierarchical loading significantly boosts efficiency in scenarios where purely history-based prediction falls short. Crucially, while absolute latency reductions in these less structured domains (e.g., Science and Business) may appear incremental compared to state-of-the-art baselines like MoE-Infinity, this margin is critical for real-world deployments on memory-constrained edge devices. In autoregressive generation, even a marginal per-token I/O overhead reduction accumulates significantly over long conversational turns. By providing a much tighter upper bound on worst-case tail latencies, ADEPT ensures the responsiveness essential for maintaining interactive user experiences.

Memory Usage: The memory footprint of the locality-aware strategy is a small fraction of the cost of full expert loading. Fully loading all experts would occupy approximately 38 GB of GPU memory (100% relative usage). By contrast, our method keeps only the most relevant experts in VRAM at any time, thanks to domain-aware loading and LRU eviction of unused experts. In our experiments, the peak number of experts concurrently resident under caching was about 43 per layer (out of 64), which corresponds to roughly two-thirds of the full model's expert memory. In other words, ADEPT uses only 67% of the GPU memory that the full-load approach would require, a substantial saving. In many cases the memory usage is even lower; the macro-average concurrent usage was about 33% of the full model size. This is the "price" paid for caching – a moderate increase in memory usage compared to an on-demand strategy – but it is far outweighed by the latency benefits. The on-demand offloading baseline, by loading only the currently needed expert and immediately discarding it, can in theory achieve the lowest memory usage. In practice, we found that even the on-demand method eventually loads a modest fraction of experts (especially for longer prompts), reaching around 10–15% of the full model size in active memory at peak. Thus, compared to on-demand, our method uses slightly more GPU memory (an increase of on the order of 20 percentage points in relative usage) in order to cache likely-future experts. Crucially, this extra memory usage yields a dramatic latency reduction (from +80% overhead down to +4% overhead in prefill, as noted above). Put another way, our strategy "spends" a bit of VRAM to save a lot of time, whereas the on-demand strategy saves a bit more memory but pays a heavy penalty in time. We consider this a very favorable trade-off for realistic deployment scenarios. In summary, the proposed locality-aware offloading nearly halves inference latency relative to on-demand loading while using only about two-thirds of the memory of full loading – demonstrating that we can *have the best of both worlds* (sparse computation with low latency) by intelligently caching just the right experts.

F. GENERALIZATION AND CROSS-DOMAIN ROBUSTNESS

We extend evaluation along two axes: (i) *cross-model* generalization across heterogeneous MoE architectures, and (ii) *cross-domain* robustness under domain misclassification. To avoid scale bias, we report relative percentages against appropriate baselines.

TABLE 12. Cross-model comparison, relative to DeepSeek (DeepSeek=100). Absolute baselines: Top-3=71.9%, TPT=1.39.

Model	Top-3 Hit (rel., %)	TPT (rel., %)
DeepSeek-MoE	100.0	100.0
Qwen1.5-MoE	82.0	107.9

TABLE 13. Cross-domain prefill hit-rate (relative, %). Diagonal=100 (in-domain).

Actual	CODE	MATH	SCI	BUS	CRE
CODE	**100**	83	84	66	71
MATH	87	**100**	77	58	68
SCIENCE	87	84	**100**	62	76
BUSINESS	76	67	62	**100**	68
CREATIVE	82	77	75	64	**100**
Avg misclass. (row-normalized): 74%			*Loss: -26%*		

1) CROSS-MODEL (RELATIVE TO DEEPSEEK)

Both **DeepSeek-MoE 16B** and **Qwen1.5-MoE A2.7B** employ 64 experts but differ in routing (Top-6 vs. Top-4) and capacity. We re-evaluate under a unified **Top-3 hit-rate** with the same **Random-Forest** predictor. The table reports values *relative to DeepSeek* (DeepSeek=100).

Under this unified criterion, Qwen retains **82%** of DeepSeek's Top-3 accuracy while incurring only **+7.9%** TPT (1.50 vs. 1.39). This confirms that ADEPT's two-stage offloading and domain-aware caching deliver consistent benefits across model scales.

2) CROSS-DOMAIN (RELATIVE TO IN-DOMAIN PER ROW)

To quantify sensitivity to domain priors, we preload Stage 1 experts from a predicted domain while evaluating prompts from the actual domain. Each cell shows the *prefill hit-rate relative to the in-domain baseline of that row* (diagonal=100). Thus, values below 100 indicate loss vs. correct classification.

On average, misclassification preserves **74%** of in-domain prefill hit-rate (i.e., **-26%** relative loss). Low-overlap pairs (e.g., business–science) show the largest drops, while high-overlap technical pairs (code–math) are comparatively robust. This aligns with Fig. 1's expert-overlap structure and highlights that accurate domain identification is a first-order lever for prefill efficiency.

G. COMPLEXITY AND OVERHEAD ANALYSIS

A primary concern in layer-wise offloading is the potential overhead introduced by the prediction mechanism. We analyze the computational complexity of our approach to demonstrate its efficiency.

TABLE 14. Time complexity of ADEPT operations.

Operation	Complexity	Measured Time
Stage 1: Domain Identification (One-time)		
Embedding (GTE-base)	$O(L \cdot H^2)$	≈ 4 ms
Medoid Search	$O(C \cdot D_{emb})$	< 1 ms
Stage 2: Expert Prediction (Per-Layer)		
Feature Extraction	$O(1)$	$<1\mu$s
RF Inference	$O(T \cdot D)$	$\sim 10\mu$s
Top-K Selection	$O(E \log E)$	$\sim 5\mu$s
Cache Management	$O(1)$	$<1\mu$s
Total per Layer	-	$\sim 20\mu$s

L: prompt length, H: hidden size, C: number of clusters. T=50 trees, D=10 depth, E=64 experts.

1) CACHING OPERATIONS COMPLEXITY

Table 14 summarizes the time complexity of our caching operations. The GPU cache is implemented using Python's OrderedDict, which combines a hash table with a doubly-linked list to achieve $O(1)$ complexity for all cache management operations.

The total per-layer overhead of $O(E \log E + T \cdot D)$ translates to approximately 20μs, which is three orders of magnitude smaller than the MoE layer computation time (10–100ms). This confirms that the caching logic introduces negligible overhead.

2) SPACE COMPLEXITY

The space overhead consists of: (i) GPU cache storing C experts of size S each, yielding $O(C \cdot S)$; and (ii) LRU metadata requiring $O(C)$ additional pointers. With $C = 43$ and $S \approx$590MB, the cache occupies approximately 25GB, representing 67% of full model memory.

3) PREDICTOR INFERENCE COST

We deliberately chose a Random Forest (RF) predictor over deep neural networks (e.g., LSTM or Transformer-based routers) to minimize CPU latency. While deep learning predictors involve computationally intensive matrix multiplications ($O(d^2)$), our RF predictor relies on simple decision tree traversals with a complexity of $O(T \cdot D)$, where T is the number of trees and D is the maximum depth. Empirically, the inference time for our RF model is approximately $10\mu s$ per token, which is orders of magnitude faster than the GPU execution time of an MoE layer ($10 \sim 100ms$).

4) FEATURE EXTRACTION EFFICIENCY

The features used for prediction (Eq. 5), such as `prev_expert` and `domain_id`, are scalar values directly available from the runtime metadata. Extracting these features requires $O(1)$ constant time operations, avoiding the high overhead of embedding lookups or sequence processing required by complex history-based predictors. Note that while the embedding generation for domain identification (Stage 1) takes approximately 4 ms (as shown in Table 14), this is a one-time cost incurred only during the prefill phase and does not affect the per-token decoding latency.

5) END-TO-END LATENCY HIDING

Furthermore, the overhead is mitigated by the asynchronous system design. The CPU predictor prefetches experts for Layer $i+1$ concurrently while the GPU computes Layer i. Consequently, the prediction latency is effectively hidden and does not add to the end-to-end critical path, provided that the prediction time is shorter than the GPU computation time.

V. DISCUSSION AND CONCLUSION

While our proposed two-stage offloading significantly reduces inference latency, we identify specific computational overheads and failure scenarios where the performance gain may be limited.

The introduction of the Random Forest predictor and Domain Classifier adds a computational step on the CPU. However, our measurements show that this overhead is negligible ($<$ 1% of the total inference time). Since the predictor runs asynchronously or in parallel with the GPU computation of the previous token, it rarely blocks the critical path. The latency reduction from avoiding PCI-e transfer far outweighs this minimal CPU cost.

Our system relies on temporal locality and domain context (Equation 5). In scenarios where the conversation topic shifts abruptly (e.g., a user switches from a "Python coding" query to a "History" question), the predictor may initially rely on the stale context of the previous domain. This can lead to a temporary increase in cache misses (false positives) for the first few tokens of the new turn until the `prev_expert` and `domain_id` features stabilize.

A. LIMITATIONS

The domain classifier is trained on a fixed set of clusters derived from the training corpus. Consequently, for queries belonging to essentially unseen domains (Out-Of-Distribution data) or highly ambiguous contexts, the domain scoring ($\mu \cdot f_{\text{domain}}$) may become less effective. In such cases, the system performance gracefully degrades, relying more on spatial and temporal features, potentially converging to the performance of standard LRU caching. Furthermore, constructing the domain-to-expert heatmaps requires offline expert-activation logging from domain corpora. This offline profiling introduces an initial computational cost and assumes that the offline corpora accurately represent the target deployment workloads. Future work could explore the online adaptation of these heatmaps to alleviate this offline dependency.

MoE models offer the promise of extreme scale with efficient, sparse computation, but their practicality has been limited by the memory–latency trade-offs of expert offloading. In this work, we showed that by exploiting locality in expert usage, it is possible to realize most of MoE's performance benefits *without* the waste of keeping every expert in GPU memory. The proposed locality-aware

offloading strategy adaptively balances memory and speed: it avoids loading unused experts (drastically reducing memory requirements) yet ensures that the truly needed experts are available on-demand from the GPU cache (minimizing latency).

Our results demonstrate that this approach can substantially improve MoE deployability. In our experiments, simulation results confirmed a 50% reduction in I/O latency, while real-world validation demonstrated a 3.45× end-to-end speedup (1100 ms vs. 3797 ms) over standard offloading baselines while using only a fraction of the GPU memory that a full expert-resident model would require. This effectively brings MoE inference closer to the latency and memory profile of a standard dense model, enabling large MoE LMs to be served in real-time on a single commodity GPU. These gains have immediate practical implications: for example, memory-constrained environments like cloud inferencing on cheaper GPU instances, or real-time applications on edge devices, can now leverage large MoE models without suffering intolerable latency [35]. By exploiting MoE's inherent sparsity more intelligently, we make it a more viable choice for latency-critical deployments.

Looking forward, our work opens up several avenues for future research and system development. First, while our study focused on a single-GPU setup, integrating locality-aware expert caching with model-parallel or multi-GPU frameworks is a promising direction [36]. Coordinated caching across multiple devices could unlock even larger MoE models or further reduce latency via parallelized expert loading. Second, there is potential to apply more sophisticated machine learning methods to the caching policy itself. In this work, we used a random forest classifier with engineered features to predict expert usage; an interesting extension would be to train a reinforcement learning agent or neural network to dynamically learn the optimal experts to load/evict over time [37]. Such an approach could adapt to non-stationary workloads or learn complex patterns beyond our current feature set. Third, our notion of prompt domain locality suggests benefits beyond MoE gating – for instance, grouping similar queries to reuse computation (decoder hidden states, attention caches, etc.) in a traditional Transformer inference pipeline. Exploring "prompt clustering" more broadly could yield system optimizations for dense models as well.

In conclusion, we have presented a practical solution to the memory–latency bottleneck of MoE inference by intelligently orchestrating which experts to load *when*. By leveraging temporal recency, cross-layer correlations, and prompt semantics, our approach achieves low-latency, memory-efficient inference on a large MoE model – substantially closing the gap between ideal performance and the constraints of limited hardware. We hope that this work encourages wider adoption of MoE models in latency-sensitive applications and inspires further research at the intersection of model design and systems optimization for large-scale AI deployments. The ability to dynamically manage sparse model components at inference time pushes the envelope of serving large models, bringing us closer to deploying trillion-parameter expert systems in everyday settings.

Due to ongoing software registration processes associated with this project, the core algorithms and simulator configurations are detailed within the manuscript in lieu of a full public code release to support reproducibility.

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.

[3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," 2017, *arXiv:1701.06538*.

[4] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.

[5] M. Lewis, S. Bhosale, T. Dettmers, N. Goyal, and L. Zettlemoyer, "BASE layers: Simplifying training of large, sparse models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021, pp. 1–10.

[6] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," 2020, *arXiv:2006.16668*.

[7] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Comput.*, vol. 3, no. 1, pp. 79–87, Mar. 1991.

[8] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on GPU clusters using megatron-LM," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2021, pp. 1–15.

[9] M. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, "Deep contextualized word representations," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, vol. 1, 2018, pp. 2227–2237.

[10] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Online, 2020, pp. 7871–7880.

[11] L.-H. Luo, "MoE-ERAS: Expert residency aware scheduling for efficient MoE inference," in *Proc. 4th Conf. Mach. Learn. Syst. (MLSys)*, Miami, FL, USA, 2023.

[12] R. Pope, S. Li, M. Patwary, D. Schoop, M. Yusti, V. Korthikanti, R. Cheng, N. Schryver, and Z. Tang, "Efficiently scaling transformer inference," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023, pp. 1–19.

[13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Chen, I. Stoica, and J. E. Gonzalez, "vLLM: Easy, fast, and cheap llm serving with pagedattention," in *Proc. 29th Symp. Operating Syst. Princ.*, 2023, pp. 560–576.

[14] DeepSeek Team. (2024). *DeepSeek-MoE: Scaling Large Language Models With Mixture-of-Experts*. Accessed: Jun. 19, 2025. [Online]. Available: https://github.com/deepseek-ai/DeepSeek-MoE

[15] B. Zoph, I. Bello, S. Kumar, N. Du, Y. Huang, J. Dean, N. Shazeer, and W. Fedus, "ST-MoE: Designing stable and transferable sparse expert models," 2022, *arXiv:2202.08906*.

[16] N. Du et al., "GLaM: Efficient scaling of language models with mixture-of-experts," 2021, *arXiv:2112.06905*.

[17] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," 2019, *arXiv:1910.02054*.

[18] E. Frantar and D. Alistarh, "QMoE: Practical sub-4-bit quantization for mixture-of-experts," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 10488–10505.

[19] Y. Sheng, S. Zhuang, L. Z. Li, W. Kwon, I. Stoica, and J. E. Gonzalez, "Sarathi: Efficient LLM inference by piggybacking preemption with pausing," in *Proc. 29th Symp. Operating Syst. Princ.*, 2023.
[20] E. Ames, H. Andréasson, and O. Rinne, "On the hoop conjecture and the weak cosmic censorship conjecture for the axisymmetric einstein-vlasov system," 2023, *arXiv:2305.04360*.
[21] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Zhao, A. Dai, Z. Chen, Q. Le, and J. Laudon, "Mixture-of-experts with expert choice routing," 2022, *arXiv:2202.09368*.
[22] Y. Zhang, T. M. Bartley, M. Graterol-Fuenmayor, V. Lavrukhin, E. Bakhturina, and B. Ginsburg, "A chat about boring problems: Studying GPT-based text normalization," 2023, *arXiv:2309.13426*.
[23] C. Riquelme, "Vision MoE: Scaling mixture of experts for efficient transfer learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021.
[24] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," 2022, *arXiv:2211.17192*.
[25] Z. Yang, X. Lu, Z. Li, P. J. Chen, F. Wang, I. Stoica, and J. E. Gonzalez, "Lookahead decoding: An exploration of what is being speculated," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024.
[26] M. Mazur, "MoE-infinity: Scaling sparsely activated models to infinity," in *Proc. 37th Conf. Neural Inf. Process. Syst. (NeurIPS)*, New Orleans, LA, USA, 2023.
[27] K. Cai, "Medusa: Simple framework for accelerating transformer inference," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023.
[28] W. Zhang, "Recurrent drafting for fast autoregressive inference," in *Proc. Findings Assoc. Comput. Linguistics (ACL)*, 2024.
[29] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention-2: Faster attention with better parallelism and memory usage," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.
[30] S. Mukherjee, A. Mitra, G. Jawahar, S. Agarwal, H. Palangi, and A. Awadallah, "Orca: Progressive learning from complex explanation traces of GPT-4," 2023, *arXiv:2306.02707*.
[31] Z. He, S. Rajbhandari, and Z. Li, "Tutel: Adaptive mixture-of-experts for efficient inference," in *Proc. Mach. Learn. Syst. (MLSys)*, 2022.
[32] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. San Mateo, CA, USA: Morgan Kaufmann, 2017.
[33] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
[34] S. Gugger, L. Debut, T. Wolf. (2022). *Accelerate: Training and Inference at Scale Made Simple, Efficient and Adaptable*. [Online]. Available: https://github.com/huggingface/accelerate
[35] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, "EdgeMoE: Empowering sparse large language models on mobile devices," *IEEE Trans. Mobile Comput.*, vol. 24, no. 8, pp. 7059–7073, Aug. 2025.
[36] Y. L. Li, A. Bapna, O. Firat, M. Chen, T. Dao, and A. W. Chen, "Branch-train-merge: Embarrassingly parallel training of expert models," in *Proc. Int. Conf. Mach. Learn.*, 2022.
[37] J. Chen, B. Yu, and D. Yu, "Fast, controllable, and interpretable generation of high-fidelity audio with latent diffusion models," *IEEE/ACM Trans. Audio, Speech, Language Process.*, 2023.

HANGYEOL KIM is currently pursuing the M.S. degree with the Department of Electrical Computer Engineering, Sungkyunkwan University, Suwon, South Korea. Since 2021, he has been a Researcher with Korea Electronics Technology Institute (KETI), where his career began with research and development on Kubernetes-based orchestration. He later worked as a Server Developer and Researcher, focusing on frameworks for providing AI workloads on edge servers. More recently, he has been conducting research on model offloading in mixture-of-experts (MoE) large language models (LLMs).

HONGUK WOO (Member, IEEE) received the B.S. degree in computer science from Korea University, Seoul, in 1995, and the M.S. and Ph.D. degrees in computer science from The University of Texas at Austin, Austin, TX, USA, in 2002 and 2008, respectively. From 2008 to 2018, he was with Samsung Research and Samsung Electronics as a Principal Engineer and the Vice President. Since 2018, he has been with the Department of Computer Science and Engineering, Sungkyunkwan University, Suwon, South Korea, and has been an Associate Professor, since 2022. His research interests include intelligent cyber-physical systems, self-managing systems, and machine learning.

YOUNGHWAN KIM received the Ph.D. degree in engineering from Soongsil University, Seoul, South Korea. Since 2003, he has been with Korea Electronics Technology Institute (KETI), where he is currently a Team Leader and a Principal Researcher with the Intelligent IDC Group. His research interests include cloud computing infrastructures (server and storage systems), virtualization middleware, embedded systems, and BMC firmware development.